efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

CHAPITRE 3 : SQL Langage de Description et de Manipulation de Données

TI404 – Bases de données 1 : Concepts de base

Coordinatrice du cours: Lena TREBAUL
lena.trebaul@efrei.fr

Du modèle logique au modèle physique

- **Client**(id_client,…)
- **Commande**(id_commande,…)
- **Contenir**(#id_commande, #id_produit)
- **Produit**(id_produit,…)

- **CREATE TABLE Client (**
  …);
- **CREATE TABLE Commande (**
  …);

- Les tables correspondant aux relations sont créées avec un langage spécifique : le **SQL**
- La syntaxe peut être (un peu) différentes dans différents SGBD, il faut donc **choisir le SGBD** avant d'écrire l'implémentation en SQL
- Les tables créées avec leurs contraintes représentent le **Modèle Physique de Données (MPD)**

Partie 1 : Structured Query Language (SQL)

SQL

- **SQL (Structured Query Language) :** langage utilisé pour gérer des données, adapté pour les données structurées et donc les BD relationnelles. Il peut être utilisé dans les BD non-relationnelles, souvent avec des modifications dans la syntaxe.

- Il peut être divisé en sous-langages :

  - SQL **Data Definition Language (DDL)** : requêtes permettant de **définir** les tables d'une BD

  - SQL **Data Manipulation Language (DML)** : requêtes permettant d'**insérer**, de **supprimer** ou de **modifier** des données

  - SQL **Data Query Language (DQL)** : requêtes permettant d'**interroger** la BD

SQL

- C’est **un langage déclaratif** : on dit **ce qu’on veut faire**, pas comment est-ce qu’on le fait

Exemple :

- En *C*, pour accéder aux valeurs sur la colonne 3 qui correspond à celle des prénoms dans un tableau T, parcourt les lignes du tableau et on récupère à chaque fois la valeur de colonne d’indice 3

- En *SQL*, on formulerait « sélectionne tous les prénoms de la table T »

Modèle physique de données (MPD)

- Le **Modèle Physique de Données (MDP)** correspond à l'implémentation du MLD et précise principalement :
  - **La structure des tables :** avec la liste des colonnes entrées dans un ordre précis
  - **Les contraintes d'intégrité :** garantissent la cohérence des données (clés, unicité, non-nullité,…)
  - **Les contraintes de validation :** garantissent l'adéquation des données avec des règles métier
    - *Ex* : vérifier qu'un email contient un @, ne pas autoriser les mots de passe de moins de 8 caractères,…
- Les contraintes précisées au niveau du SGBD vont s'appliquer à toutes les applications utilisant la BD

SQL : quelques éléments pour débuter

```sql
-- Création d'une nouvelle base de données
create database bd_cinema;
-- Activation de la base de données
use bd_cinema;

-- Visualiser les tables d'une base de données
show tables;
-- Visualiser la description d'une table
describe cinema;
```

Conventions SQL

- Par convention, une instruction SQL se termine toujours par un point-virgule **";**
- Les mots-clés du langage doivent être facilement repérés par rapport au nom des tables ou attributs (utilisez des majuscules si le SGBD ne met pas en valeur les mots-clés)
- Double tiret **"—"** ou **"/* */"** indiquera une ligne ou un bloc de commentaire
- **'**string**'** Un guillemet simple délimitera une chaine de caractères.
- Comme dans tout langage de programmation, une indentation efficace assurera une lecture simple et rapide. Les bonnes pratiques vues dans les autres cours s'appliquent ici aussi, **aérer** son code, mettre des **commentaires**.

Source officielle SQL (recommandée) : https://sql.sh/

Partie 2 : SQL Langage de Définition de Données (LDD)

CREATE, ALTER et DROP

LDD : Présentation

- Sous-ensemble du langage SQL
- Manipuler les **structures de données** et non les données elles-mêmes
- Permet de :
  - Créer les **tables** avec la liste des attributs obtenus dans le MLD
  - Définir le **domaine** des données
  - Définir les **liens** entre plusieurs tables en utilisant les clés du MLD
  - Ajouter des **contraintes** de valeur sur les données

Contraintes d'intégrité du Modèle Relationnel

- **Intégrité de domaine :** la valeur de chaque attribut appartient à un domaine valide
  - Il correspond à un type de données (age : INT)

- **Intégrité d'entité :** chaque relation doit avoir une **clé primaire**
  - La clé primaire a une valeur unique
  - La clé primaire a une valeur non-nulle

- **Intégrité référentielle** : une **clé étrangère** doit correspondre à une clé primaire existante *(ou être nulle si c'est autorisé)*
  - Un film est réalisé par un réalisateur d'ID 5 -> l'ID 5 doit exister dans la table des réalisateurs

Les types SQL (Norme ANSI)

	Type	Description	Taille
entiers	INTEGER	Type des entiers relatifs	4 octets
	SMALLINT	Idem.	2 octets
	BIGINT	Idem.	8 octets
flottants	FLOAT	Flottants simple précision	4 octets
	DOUBLE PRECISION	Flottants double précision	8 octets
	REAL	Synonyme de FLOAT	4 octets
nombres à précision fixe	NUMERIC (*M*, *D*)	Numérique avec précision fixe.	*M* octets
	DECIMAL (*M*, *D*)	Idem.	*M* octets
caractères	CHAR(*M*)	Chaînes de longueur fixe	*M* octets
	VARCHAR(*M*)	Chaînes de longueur variable	$L+1$ avec $L \leqslant M$
	BIT VARYING	Chaînes d'octets	Longueur de la chaîne.
date/horaire	DATE	Date (jour, mois, an)	env. 4 octets
	TIME	Horaire (heure, minutes, secondes)	env. 4 octets
	DATETIME	Date et heure	8 octets
	YEAR	Année	2 octets

Format :

'AAAA-MM-JJ'
'HH:MM:SS'
'AAAA-MM-JJ HH:MM:SS'

Création des tables : CREATE TABLE

Exemple : **Internaute**

MLD : Internaute(**email**, nom, prenom, mdp, année_naissance)

SQL

```
create table internaute(
email varchar(50) primary key,
nom varchar(50) not null,
prenom varchar(50) not null,
mdp varchar(50) not null,
anne_naissance int);
```

- Chaque attribut déclaré correspond à une nouvelle colonne dans la table
- Chaque attribut à un type, on peut lui ajouter une contrainte ensuite comme PRIMARY KEY, UNIQUE, NOT NULL…
- Chaque table doit avoir un ou plusieurs attributs déclarés en tant que **la** clé primaire (même si le SGBD peut ne pas déclencher d'erreur si ce n'est pas le cas)

Contrainte d'intégrité : clé primaire

- La contrainte **PRIMARY KEY** peut se mettre à la suite du nom de l'attribut utilisé ou sur une ligne séparée
- Si la clé primaire est formée de plusieurs attributs, il faut ajouter la contrainte sur une nouvelle ligne

```
create table internaute(
    email varchar(50) not null,
    nom varchar(50) not null,
    prenom varchar(50) not null,
    mdp varchar(50) not null,
    anne_naissance int,
    primary key(email));
```

Ou encore

```
create table internaute(
email varchar(50) primary key,
nom varchar(50) not null,
prenom varchar(50) not null,
mdp varchar(50) not null,
anne_naissance int);
```

```
create table notation(
id_film int not null,
email varchar(50) not null,
note int default 0,
primary key(id_film, email));
```

Contraintes : NOT NULL, UNIQUE et DEFAULT

MLD : **artiste(<u>id_artiste</u>, nom, prenom,** annee_naissance) SQL

```
create table artiste(
    id_artiste int not null,
    nom varchar(50) not null,
    prenom varchar(50) not null,
    annee_naissance int default 2000,
    primary key(id_artiste),
    unique(nom, prenom));
```

- **NOT NULL** permet de s'assurer que l'on aura une valeur pour cet attribut pour chaque ligne insérée dans la table.
- Cela évite des problèmes notamment par rapport aux calculs/comparaisons qui ne peuvent pas être faits avec des valeurs nulles.

- On peut spécifier que la **valeur** d'un attribut ou une combinaison d'attributs est **unique** pour l'ensemble de la colonne avec **UNIQUE**. Cela permet d'indiquer des **clés secondaires**.
- La clause **UNIQUE** ne s'applique pas aux valeurs NULL si elles sont permises : elles peuvent se répéter.

- La contrainte **PRIMARY KEY** vérifie déjà que les valeurs seront uniques et non-nulles.

- Ajouter **DEFAULT** permet d'attribuer une valeur par défault à l'attribut si aucune valeur n'est entrée.

Contrainte d'intégrité référentielle : clé étrangère

Une **clé étrangère** est spécifiée avec l'option **FOREIGN KEY**

MLD : **film**(**id_film**, titre, année, #id_mes, #code_pays)

SQL

```
create table film(
    id_film int not null,
    titre varchar(50) not null,
    annee int not null,
    id_mes int,
    code_pays int,
    primary key(id_film),
    foreign key(id_mes) references artiste(id_artiste),
    foreign key (code_pays) references pays(code_pays));
```

Le SGBD vérifie ***pour toute modification*** pouvant affecter le lien entre les deux tables, que la valeur de ID_MES correspond bien à une ligne de ARTISTE

Lors de :

- ✓ L'**insertion** dans FILM avec une valeur inconnue pour ID_MES
- ✓ La **suppression** d'un artiste
- ✓ La **modification** de ID_ARTISTE dans ARTISTE ou de ID_MES dans FILM

Note : La contrainte t'intégrité ne s'applique pas aux valeurs nulles : *id_mes* n'est pas contraint par la clause NOT NULL, dans ce cas NULL est accepté et ne viole pas la contrainte d'intégrité.

Gérer la violation de la contrainte de clé étrangère

S'il y a **violation d'une contrainte d'intégrité** :

- La mise à jour est **rejetée par défaut**, mais il est possible de demander la répercussion de cette mise à jour de manière à ce que la contrainte soit respectée.
- Les événements que l'on peut répercuter sont la **modification** ou la **destruction** de la ligne référencée, et on les désigne par **ON UPDATE** et **ON DELETE** respectivement.
- La répercussion elle-même consiste soit à **mettre la clé étrangère à NULL** (option **SET NULL**), soit à **appliquer la même opération** aux lignes de l'entité composante (option **CASCADE**).

```
create table film(
    id_film int not null,
    titre varchar(50) not null,
    annee int not null,
    id_mes int,
    code_pays int,
    primary key(id_film),
    foreign key(id_mes) references artiste(id_artiste)
    on delete set null,
    foreign key (code_pays) references pays(code_pays));
```

Intégrité référentielle : cas des entités faibles/fortes

Dans le cas d'une entité faible, si on accepte de détruire la table qui correspond à l'entité forte, alors il faut détruire la table qui correspond à l'entité faible.

- On détruit le composant quand on détruit le composé.

Exemple : « Quand on détruit un cinéma, on veut également détruire ses salles; quand on modifie la clé d'un cinéma, on veut répercuter la modification sur ses salles »

```
create table salle
    nom_cinema varchar(30) not null,
    num_salle int not null,
    capacite int,
    primary key (nom_cinema, num_salle) ,
    foreign key(nom_cinema) references cinema(nom_cinema)
    on delete cascade
    on update cascade);
```

Contraintes de validation : CHECK

Énumération des valeurs possibles avec **CHECK** (condition) :

- Permet d'exprimer des **contraintes** portant soit sur un attribut, soit sur une ligne
- Les contraintes les plus courantes sont celles consistant à restreindre un attribut à un ensemble de valeurs

Exemple dans un CREATE TABLE :

```
create table film(
    id_film int not null,
    titre varchar(50) not null,
    annee int check (annee between 1890 and 2000) not null,
    id_mes int,
    code_pays int,
    primary key(id_film),
    foreign key(id_mes) references artiste(id_artiste)
    on delete set null,
    foreign key (code_pays) references pays(code_pays));
```

Exemples d'expressions :

```
CHECK (age >= 18)
CHECK (prix > 0)
CHECK (date_fin > date_debut)
CHECK (note BETWEEN 0 AND 20)
CHECK (statut IN ('en_cours', 'termine', 'annule'))
```

ALTER TABLE : modification du schéma

ALTER TABLE permet de modifier :

- Le schéma d’une table

- Une colonne

Mais aussi d’ajouter **des contraintes** sur la table quand c’est nécessaire

ALTER TABLE nomTable **ACTION** description

– **ACTION** : ADD, MODIFY, ALTER, DROP ou RENAME

– **Description** : commande de modification associée à ACTION

ALTER TABLE : quelques exemples

1- **Ajouter** des attributs

```
alter table internaute add region varchar(15);
```

S'il existe déjà des données dans la table, la valeur sera à NULL ou à la valeur par défaut.

2- **Modifier** un domaine déjà défini avec MODIFY, et **ajouter** une **contrainte** comme NOT NULL :

```
alter table internaute modify region varchar(30) not null;
```

3- **Ajouter** une valeur par **défaut** :

```
alter table internaute alter region set default 'PACA';
```

4- **Supprimer** un attribut :

```
alter table internaute drop region;
```

ALTER TABLE : ajout d'une contrainte intégrité

```sql
Create table Film(
    ID_Film INT,
    Titre_Film varchar(20),
    ID_MES INT
);

create table artiste(
    ID_Artiste INT,
    Nom_artiste varchar(20),
    Prenom_artiste varchar(20),
    primary key(ID_Artiste)
);

Alter table Film ADD constraint pk_film PRIMARY KEY (ID_Film);
Alter table Film ADD constraint Fk_film FOREIGN KEY (ID_MES) REFERENCES Artiste(ID_Artiste);
```

ALTER TABLE : ajout d'une contrainte de validation

```
CREATE TABLE Artiste (
   Nom varchar(20) NOT NULL,
   Prenom varchar(15),
   Annee_naissance decimal(4,0),
   PRIMARY KEY (Nom)
)

alter table artiste add constraint ck_anneenaissance check (Annee_naissance >= 1980);
```

ou encore

```
CREATE TABLE Cinema (
   Nom_cinema varchar(10) NOT NULL,
   Arrondissement decimal(2,0),
   Adresse varchar(30),
   PRIMARY KEY (Nom_cinema)
);
```

Option 1

```
alter table cinema add constraint ck_arrondissement check (Arrondissement between 1 and 20);
```

Option 2

```
alter table cinema add check (Arrondissement between 1 and 20);
```

Suppression : DROP

- Supprimer le schéma de la base de données :

```
drop database DB_cinema;
```

- Supprimer une table :

```
drop table cinema;
```

- Supprimer une contrainte :

```
alter table cinema drop constraint ck_arrondissement;
```

Si un nom de contrainte n’est pas rajouté dans la requête, le SGBD en attribue un automatiquement, qu’il faut retrouver si on veut la supprimer.

Partie 3 : SQL Langage de Manipulation de Données (LMD)

INSERT, UPDATE et DELETE

Insertion des données (INSERT INTO)

Insertion d'une seule ligne/enregistrement :

Bien respecter l'ordre des attributs

INSERT INTO nom_table VALUES(valeur_att1, valeur_att2,…);

INSERT INTO nom_table(att1_name, att2_name, att3_name) values (val1, val2, val3)

Insertion de plusieurs enregistrements en même temps :

INSERT INTO nom_table VALUES((valeur_att1, valeur_att2,…), (valeur_att1, valeur_att2,…), (valeur_att1, valeur_att2,…), …);

Exemple :

```
insert into cinema values('pathe', '15eme arrondissement');
insert into cinema values('rex', '10eme arrondissement');
-- ou encore
insert into cinema values(('pathe', '15eme arrondissement'), ('rex', '10eme arrondissement'));
```

Modification des données (UPDATE)

- Modification d'*une seule ligne* :

UPDATE nom_table SET att1 = nv_valeur WHERE cle_primaire = val

- Modification d'*un ensemble de lignes* :

UPDATE nom_table SET att1 = nv_valeur WHERE att2 = val

- Modification de *toutes les lignes* :

UPDATE nom_table SET att1 = nv_valeur

Certains SGBD **n'autorisent pas** par défaut la modification de lignes identifiées **sans utiliser une clé primaire**

Exemple :

```
update films set titre = 'Shining' where id_film = 404;
update films set annee = 2025 where titre = 'The Brutalist';
update films set annee = 1990;
```

Suppression des données (DELETE)

- Suppression d'*une seule ligne* :

DELETE FROM nom_table WHERE cle_primaire = val

- Suppression d'*un ensemble de lignes* :

DELETE FROM nom_table WHERE att1 = val

- Suppression de *toutes les lignes* :

DELETE FROM nom_table;

Certains SGBD **n'autorisent pas** par défaut la suppression de lignes identifiées **sans utiliser une clé primaire**

Exemple :

```
delete from films where id_film = 1;
delete from films where annee = 2020;
delete from films;
```

Démonstration :
Implémentation sur MySQL

MLD :

Client = (<u>cl_id</u> *INT*, cl_nom *VARCHAR(50)***);**

Commande = (<u>co_id</u> *INT*, co_date *DATE*, *#cl_id***);**

Prochain CHAPITRE:

Interrogation d'une BD en algèbre relationnelle et en SQL

02